

AILink 8-Electrode Body-Fat Scale

Native module integration guide — `AilinkScaleModule.kt`

caregiver-app · module `modules/ailink-scale` · Expo SDK 54 / RN 0.81 (New Architecture) · Generated 2026-07-12

1. What this module does

Bridges the AILink (elinkthings) 8-electrode body-fat scale to JavaScript as a local Expo native module named `AilinkScale`. The scale computes body composition **onboard** (MCU algorithm); the app supplies a user profile, streams live weight, and receives the finished result as a single `bodyfat` event. All device events are delivered on one emitter: `onScaleEvent`.

End-to-end flow

JS `initialize()` (bind BLE service) → JS `setUserInfo(sex,age,heightCm)` → JS `start(mac, timeout)` → connect → `onSyncUserInfo` ⇒ native replies `setUserInfo(...)` → live `weight` events → scale runs MCU algorithm → `onTestSuccess` ⇒ `bodyfat` event (weight, BMI, fat %, muscle, water, BMR, visceral, bone, body age, protein, HR).

2. File layout

```
modules/ailink-scale/
├─ expo-module.config.json      # autolink → registers the Android module
├─ android/
│   └─ build.gradle             # JitPack repo + AILink SDK deps
│       └─ src/main/
│           └─ AndroidManifest.xml # ELinkBleServer service + FGS permissions
│               └─ java/expo/modules/ailinkscale/
│                   └─ AilinkScaleModule.kt # the bridge (Module definition)
└─ (JS side lives in app) src/shared/lib/ailinkScale.ts # requireNativeModule('AilinkScale')
```

1 Scaffold the local Expo module

Create `modules/ailink-scale/` and declare it to Expo autolinking. Local modules under `modules/` with an `expo-module.config.json` are linked automatically — no entry in the app's `package.json` is needed.

expo-module.config.json

```
{
  "platforms": ["android"],
  "android": {
    "modules": ["expo.modules.ailinkscale.AilinkScaleModule"]
  }
}
```

2 Gradle dependencies (JitPack)

The AILink core SDK and its parsing/algorithm library are published on JitPack. Add the repo and both artifacts in `android/build.gradle`.

```
plugins {
  id 'com.android.library'
  id 'expo-module-gradle-plugin'
}
group = 'expo.modules.ailinkscale'
android { namespace "expo.modules.ailinkscale" }
```

```
repositories { maven { url 'https://jitpack.io' } }

dependencies {
    implementation 'com.github.elinkthings:AILinkSDKRepositoryAndroid:1.15.04'
    implementation 'com.github.elinkthings:AILinkSDKParsingLibraryAndroid:1.9.20'
}
```

Version pin — do not lower. The parsing library **must be $\geq$ 1.9.20**. Earlier builds (e.g. 1.9.13) throw `ArrayIndexOutOfBoundsException` while parsing the scale's short status packets, crashing the measurement. 1.9.20 wraps the packet parser in try/catch.

3 AndroidManifest — the bound BLE service

The AILink SDK routes *all* BLE traffic through a bound `ELinkBleServer`. It must be declared, along with the foreground-service permissions for a connected device.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE" />
    <application>
        <service
            android:name="com.pingwang.bluetoothlib.server.ELinkBleServer"
            android:foregroundServiceType="connectedDevice" />
        </application>
</manifest>
```

BLE runtime permissions (BLUETOOTH_SCAN / BLUETOOTH_CONNECT / location) are requested app-side and declared in the app's `app.json`.

4 Initialize the SDK service (async bind)

Binding `ELinkBleServer` is asynchronous. Scanning or connecting before the bind completes races and throws *"please call init()"*. So `initialize()` is exposed separately and JS calls it on screen mount, so the service is ready before the user taps Connect. The bind is posted to the **main thread**.

```
AsyncFunction("initialize") { promise: Promise ->
  val context = appContext.reactContext?.applicationContext ?: return@... reject(...)
  try { AILinkSDK.getInstance().init(context) } catch (_: Throwable) {}
  val manager = AILinkBleManager.getInstance()
  if (manager.isInitOk) { promise.resolve(true); return@AsyncFunction }
  Handler(Looper.getMainLooper()).post {
    manager.init(context, object : AILinkBleManager.onInitListener {
      override fun onInitSuccess() { promise.resolve(true) }
      override fun onInitFailure() { promise.reject("E_INIT_FAILED", ...) }
    })
  }
}
```

Main-thread + init-gate. Every SDK entry point (init, scan, connect) is dispatched via `Handler(Looper.getMainLooper())`, and work only begins after `isInitOk` / `onInitSuccess`. This resolves the init race that otherwise crashes on first connect.

5 Set the user profile

The MCU algorithm needs sex, age and height to compute body composition. JS pushes these before `start()`; the native side caches them and hands them back when the scale asks (`onSyncUserInfo`).

```
AsyncFunction("setUserInfo") { sex: Int, age: Int, heightCm: Int ->
  userSex = sex          // 1 = male, 0 = female
  userAge = age
  userHeightCm = heightCm
}
```

6 Start a session — connect by MAC or scan

`start(mac, scanTimeoutMs)` supports two paths. Preferred: JS discovers the device with the reliable `react-native-ble-plx` scanner and passes the **MAC directly** for a straight connect. Fallback: the AILink broadcast-UUID scan.

```
AsyncFunction("start") { mac: String?, scanTimeoutMs: Double, promise: Promise ->
  sessionActive = true; connecting = false; finished = false
  val manager = AILinkBleManager.getInstance()
  val begin = Runnable {
    manager.setOnCallbackBle(bleCallback)
    if (!mac.isNullOrEmpty()) {
      manager.stopScan(); manager.connectDevice(mac)          // direct connect
    } else {
      manager.startScan(scanTimeoutMs.toLong(),
        BleConfig.UUID_BROADCAST_AILINK,
        BleConfig.UUID_SERVER_BROADCAST_AILINK,
        BleConfig.UUID_SERVER_AILINK)                          // fallback scan
    }
    promise.resolve(null)
  }
  Handler(Looper.getMainLooper()).post {
    if (manager.isInitOk) begin.run()
    else manager.init(context, /* onInitSuccess → begin.run() */)
  }
}
```

Why ble-plx does discovery. The scale advertises on the Allink broadcast UUID (`F0A0`), not the classic `FFE0` . ble-plx scanning in JS proved more reliable for finding it; the native scan remains as a fallback.

7 Connection callbacks (OnCallbackBle)

- `onScanning` — emits `scanFound` (name/CID/MAC for bring-up visibility), then auto-connects the first AILink hit (scan is UUID-filtered; the scale is the only AILink device in this deployment).
- `onServicesDiscovered` — wraps the handle in `EightBodyFatMcuDeviceData`, attaches `fatCallback`, emits `connected`.
- `onDisconnected` — clears state, emits `disconnected`.
- `onScanTimeout` — emits a friendly `error` ("step on the scale to wake it up").

8 Measurement callbacks (MCU body-fat)

The onboard algorithm drives the sequence. The critical handshake is `onSyncUserInfo` — the scale **blocks** until the app answers with `setUserInfo`.

```
onSyncUserInfo() → deviceData?.setUserInfo(1, 0, userSex, userAge, userHeightCm)
onWeight(...) → emit "weight" (weightKg, stable) // live streaming
onHeartRate(...) → emit "heartRate"
onTestSuccess(info) → emit "bodyfat" { // final result
  weightKg, bmi, bodyFatPct(bfr), subcutaneousFatPct(sfr),
  visceralFat(uvi), musclePct(rom), bmr, boneMassKg(bm),
  waterPct(vwc), bodyAge, proteinPct(pp), heartRate }
onErrCode(code) → emit "error" (also Log.e)
```

If `onTestSuccess` arrives with null info, the module falls back to `emitWeightOnly()` → a `measureEnd` event carrying the last stable weight and HR. Unit conversion (`rawToKg`) normalizes JIN / LB / ST → kg.

9 Stop / teardown

```
AsyncFunction("stop") { promise ->
  sessionActive = false; connecting = false
  Handler(Looper.getMainLooper()).post {
    manager.stopScan(); manager.disconnectAll()
    manager.removeOnCallbackBle(bleCallback)
  }
  deviceData = null; promise.resolve(null)
}
```

10 JavaScript side

The app talks to the module through [src/shared/lib/ailinkScale.ts](#), which acquires the native module and subscribes to the single event stream. ble-plx handles discovery, then hands the MAC to `start()`.

```
const { requireNativeModule } = require('expo-modules-core');
const native = requireNativeModule('AilinkScale');

native.addListener('onScaleEvent', (e) => { /* weight | bodyfat | error ... */ });
await native.initialize();
await native.setUserInfo(sex, age, heightCm);
await native.start(macFromBlePlx, 30000);
// ... later
await native.stop();
```

Event reference — `onScaleEvent.type`

type	payload	meaning
note	message	status text (service ready, measuring...)

scanning / scanFound	name, cid, mac	scan progress / device seen
connecting / connected	mac	connection lifecycle
weight	weightKg, stable	live weight (stream)
heartRate	heartRate	live HR
bodyfat	full composition	final result after MCU algorithm
measureEnd	weightKg, heartRate	weight-only fallback (no composition)
disconnected	code	link dropped
error	message	scan timeout / err code / init failure

Build & run

- Native code ⇒ requires an **EAS dev build** (Expo Go cannot load custom native modules). Project: `venkatar/caregiver-app` , package `com.venkatar.caregiverapp` .
- Android only (`"platforms": [ "android" ]`).
- Grant Bluetooth + location runtime permissions; wake the scale (step on it) before scanning.

Troubleshooting checklist

Symptom	Cause / fix
Crash on measurement (<code>ArrayIndexOutOfBoundsException</code>)	Parsing lib too old — pin <code>AILinkSDKParsingLibraryAndroid:1.9.20+</code> .
" <i>please call init()</i> " on connect	Init race — ensure <code>initialize()</code> resolved (isInitOk) before <code>start()</code> ; all SDK calls on main thread.
Scale never found	It advertises on AILink UUID <code>F0A0</code> , not <code>FFE0</code> ; use ble-plx discovery and pass MAC to <code>start()</code> . Wake the scale first.
Weight but no body composition	<code>onSyncUserInfo</code> must be answered with <code>setUserInfo</code> ; set the user profile before <code>start()</code> .
Module not found in JS	Confirm <code>expo-module.config.json</code> lists the module and rebuild (autolink runs at build time).